

Introduction to Programming



W10 Programming Languages

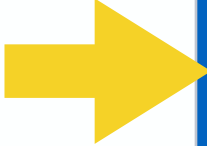
Stephan Krusche

18 December 2024
Technical University of Munich



Schedule



#	Date	Subject
1	16.10.24	Introduction
2	23.10.24	Control Flow in Programming
3	30.10.24	In-Depth Core Concepts*
4	06.11.24	Core Data Structures
5	13.11.24	Code Reuse and Structure
6	20.11.24	Type Flexibility and Safety
7	27.11.24	In-Depth Object Orientation*
8	04.12.24	Functional Programming Essentials
9	11.12.24	Algorithms and Data Handling
 10	18.12.24	Programming Languages
11	08.01.25	Graphical User Interfaces
12	15.01.25	Recursion
13	22.01.25	Concurrency
14	29.01.25	Beyond Programming
15	05.02.25	Course Review

- **Context**

- Apply OOP concepts: abstraction, encapsulation, inheritance and polymorphism
- Use control structures and data types
- Use enums, annotations, generics, collections, iterators, lambdas, and streams
- Apply error handling and implement algorithms

- **Learning goals**

- Explain the differences between compiled and interpreted languages
- Explain the differences between general purpose and domain specific languages
- Create a syntax tree for a simple program
- Model control flow diagrams
- Evaluate if words adhere to a regular expression

Tutor workshop



Jana



Jasmina



Catherine



Ema





Katherine
Johnson



Grace
Hopper



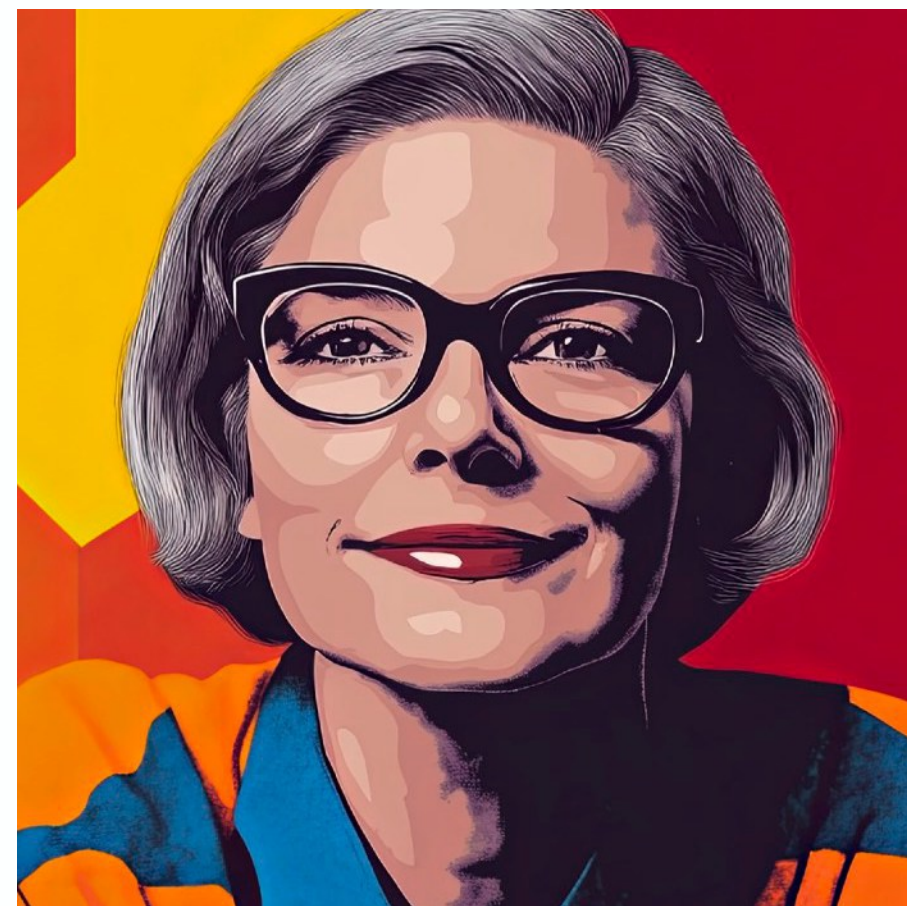
Anne
Brüggemann-Klein



Frances
Allen



Margaret
Hamilton



Christiane
Floyd



Outline

Overview

- Domain specific languages
- Syntax trees
- Control flow diagrams
- Regular expressions

What is a programming language?

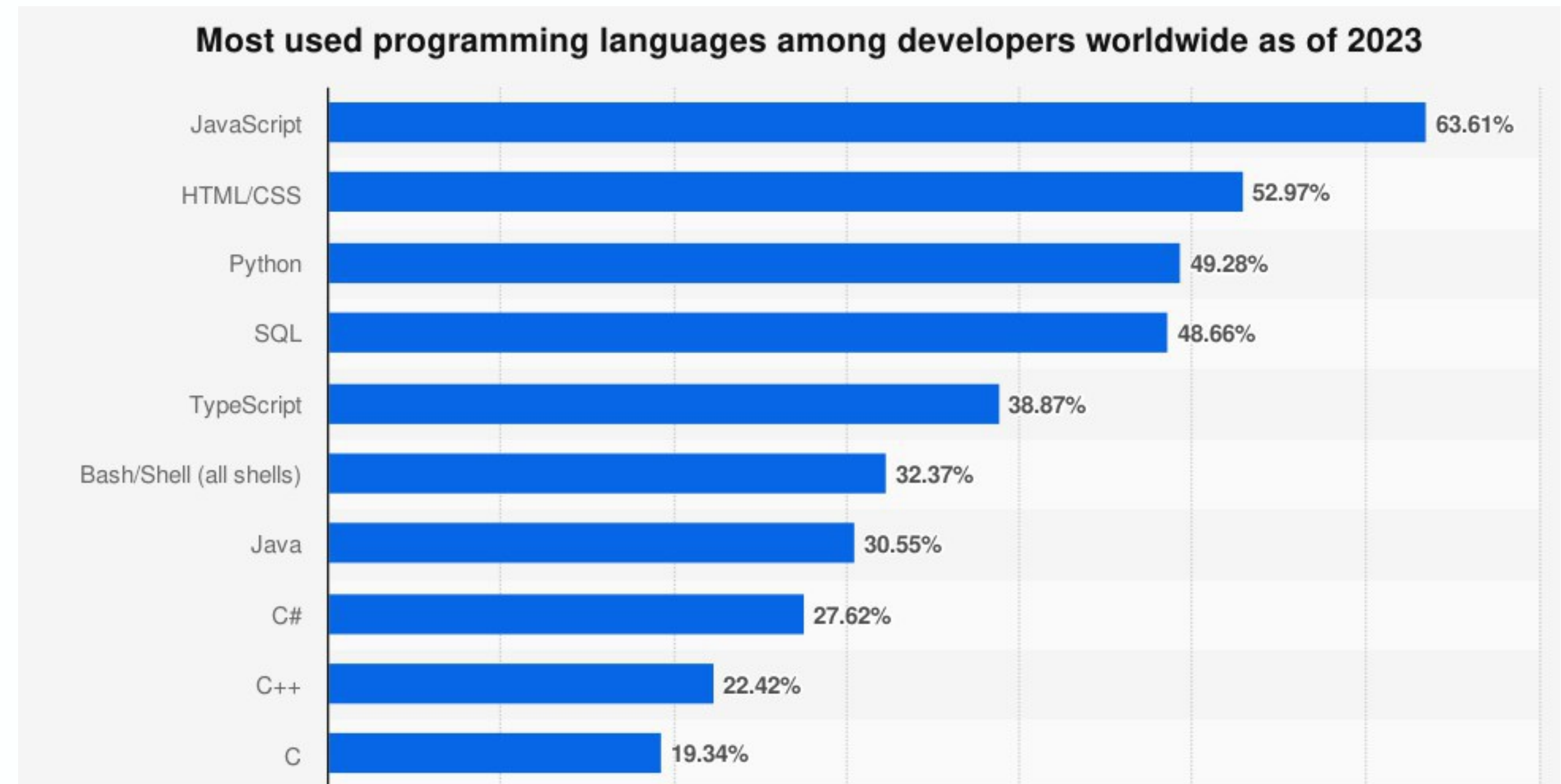


- Programming languages allow us to give instructions to a computer
- Many different human languages → many computer programming languages
- Code cannot be directly understood by a computer: it must first be translated into a lower language that the computer can execute, called a **binary**
- Translating code into binary is called **compiling** or **compilation**
- Every language, from C to Python to Java, has its own distinct features, as well as commonalities with other languages

<https://www.computerscience.org/resources/computer-programming-languages>

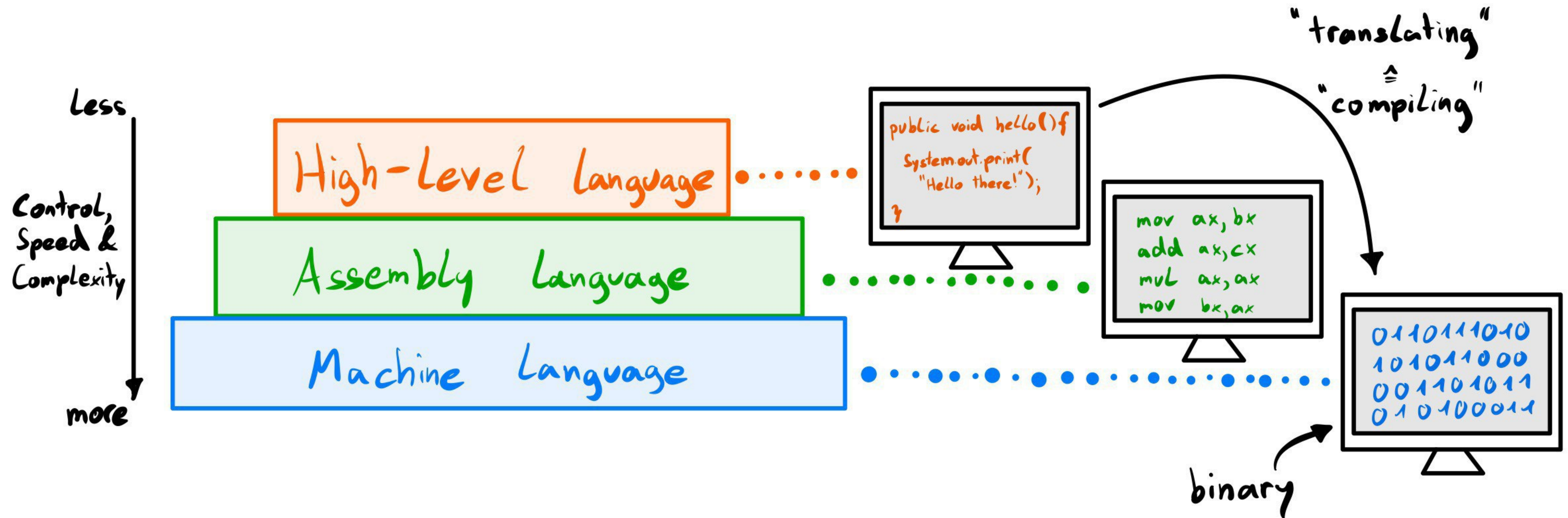
Why is there no universal language?

1. Diversity of Applications
2. Performance Requirements
3. Abstraction Levels
4. Paradigm Differences
5. Evolving Needs
6. Community and Ecosystem
7. Legacy Systems



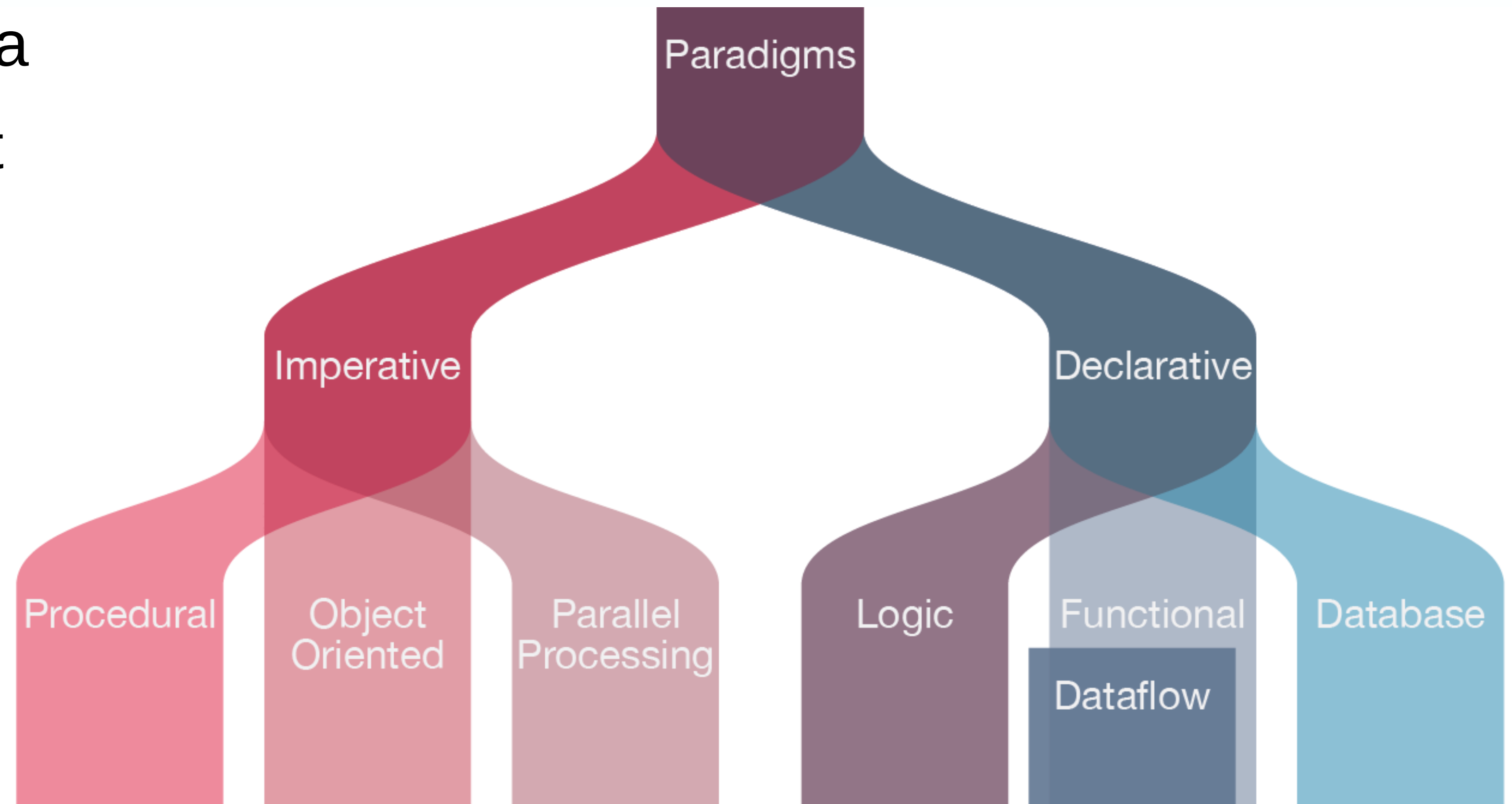
<https://www.statista.com/statistics/793628/worldwide-developer-survey-most-used-languages/>

Categories

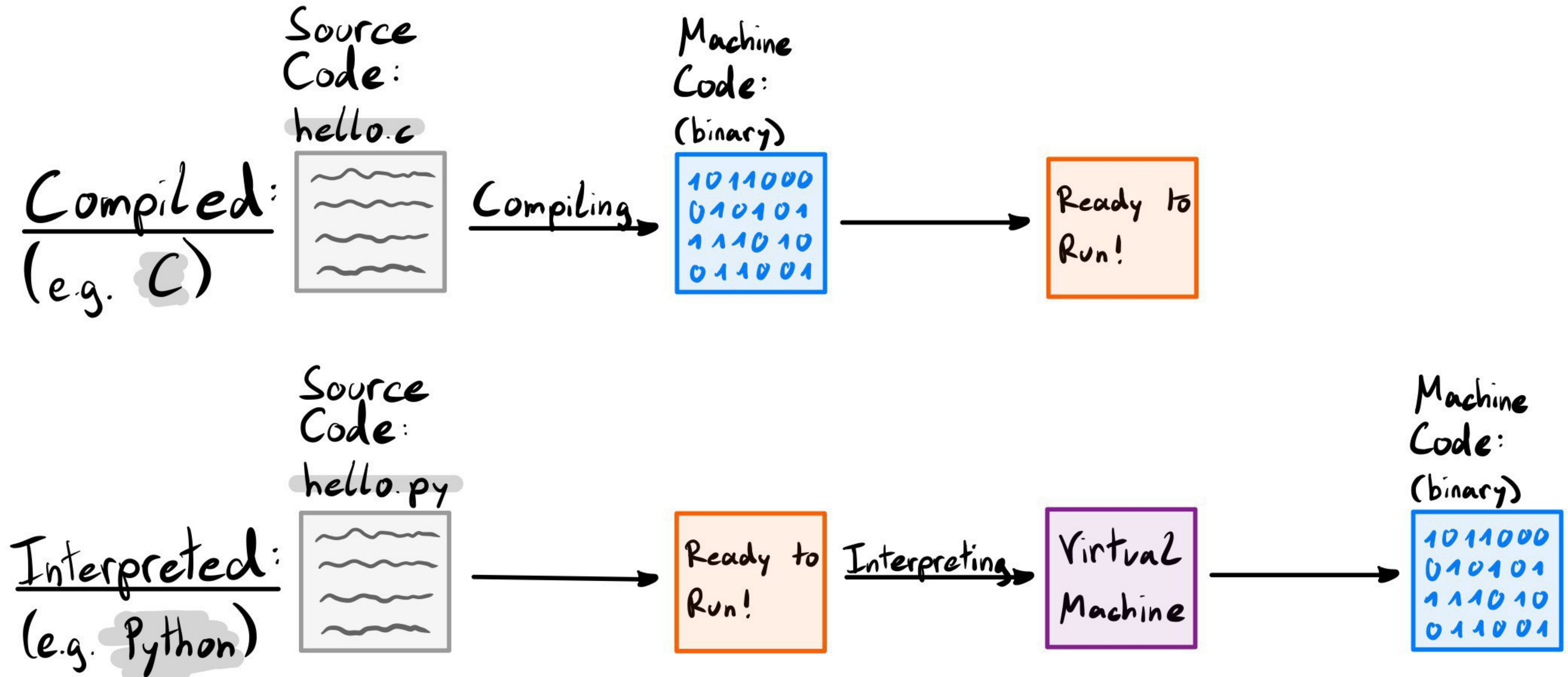


Types of high-level programming languages

- Procedural e.g. C
- Functional e.g. OCaml
- Object-oriented e.g. Java
- Scripting e.g. JavaScript
- Logic e.g. Prolog



Compiled & interpreted languages

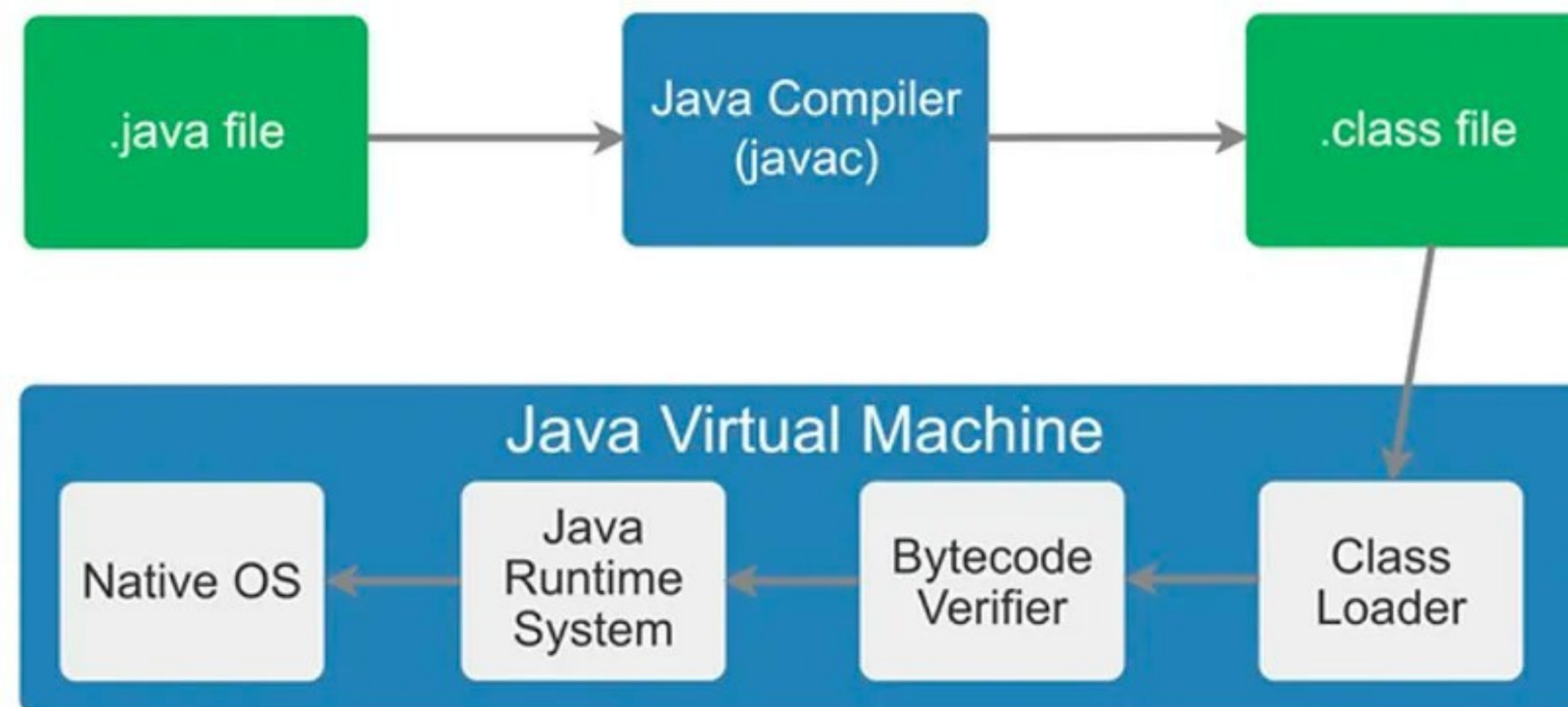


Advantages of compiled & interpreted languages



- Advantages of compiled languages
 - Faster performance due to translation to machine code in advance
 - Opportunity to apply quite powerful optimizations during the compile stage
- Advantages of interpreted languages
 - Easier to implement (writing good compilers is very hard)
 - No need to run a compilation stage: can execute code directly on-the-fly
 - Better suited for dynamic languages due to on-the-fly code changes and immediate feedback

- Unique mix: first compiled to an intermediate form (byte code), then interpreted
- Some might argue that Java is actually a compiled language, as source code gets compiled into byte code, which is a valid point
- The Java Virtual Machine (JVM) performs many optimizations: machine code is often already generated and cached → no actual interpretation going on at runtime
- For more details read: <https://www.scaler.com/topics/java/how-java-program-works/>



Byte code vs. machine code

Byte Code	Machine Code
Binary, hexadecimal and macro instructions are not understood directly by the CPU, but are dependent on an interpreter	Binary instructions that are directly understandable by the CPU and directly executed
Intermediate level Must be translated into machine code to execute	Lowest level, directly executable
Non-runnable code generated after compilation	Set of instructions in machine language
Platform-independent depends on the virtual machine	Not platform independent depends on the system architecture and native instructions associated with the machine